

AI Customer Support Platform

Architecture Overview - Plain-English Guide

This document explains what the project is, how it works, and why it was built this way - written in plain language, without assuming deep technical background. It's meant to help explain the project confidently in conversation, e.g. in an interview.

The flagship example used throughout is "Nexus Telecom" - a fictional mobile network operator built on top of the platform to demonstrate it at full strength.

1. What is this project, in one paragraph

This is a customer support chatbot platform that any company can plug their own documents into and get a working AI support agent. It is "multi-tenant", meaning one system serves many different companies (called "clients"), each kept fully separate with their own documents, branding, and data. The flagship example, Nexus Telecom, shows the platform at full strength: a chatbot that can both answer questions AND actually look up or change a customer's account, backed by a real admin dashboard that human staff use to manage those same customers.

2. The problem it solves

- Most support questions already have an answer sitting in some document (a policy, an FAQ, a price list) - but customers do not want to search a PDF, they want to ask a question and get a direct answer.
- Companies also want the bot to actually DO things, not just answer questions - check a balance, log a complaint, change a plan - but doing that safely, only for the real account owner, is the hard part.
- This project answers both needs: a chatbot that answers using the company's own real documents (instead of the AI's general knowledge, which can simply be wrong), and one that can safely take real actions on a real account, with identity checks in place.

3. Who uses it

- Customers - chat with the bot through a simple web widget or WhatsApp to ask questions and get help with their account.
- Client staff (e.g. Nexus Telecom's support team) - log into their own admin portal to see customers, tickets, subscriptions, and billing - the exact same data the bot works with.
- The platform operator (the company running the whole platform) - has one central console to add new clients, upload their documents, and monitor everything.

4. How the pieces fit together

Four main parts, working together:

- Frontend (React website) - shows three different views depending on who is visiting: the customer chat widget, the client staff portal, and the operator's console.
- Backend (Python, FastAPI) - a web server that all three views talk to. This is where the "brain" of the system lives.
- Database - stores all the structured data: which clients exist, their customers, phone lines, tickets, billing records, and so on.
- Document search (a "vector store") - a company's uploaded documents (FAQs, PDFs) are broken into small chunks and stored in a special search index (FAISS) that finds the most relevant chunk for a question by MEANING, not just matching keywords.
- AI models (Google Gemini or Groq's Llama models) - write the natural-language answer, using the relevant document chunks as their source of truth.
- Extra channels - the same bot also works over WhatsApp, and there is a live voice-call mode (speech in, speech out).

5. What happens when a customer sends a message

Step by step, in plain language:

1. The message arrives at the backend.
2. The system figures out: is this a general question, or does it need a real action (like "what is my balance")?
3. For a general question, it searches the company's documents for the most relevant pieces of text. This search-first-then-answer approach is called RAG (Retrieval-Augmented Generation): retrieve the real facts first, then have the AI write the answer using only that information - instead of letting the AI guess from its general training, which can produce confidently wrong answers.
4. For an action (checking an account, looking up call history, changing a plan), the system calls a specific, safe backend function to do exactly that - the AI never invents account data itself.
5. Security step: before touching any specific customer's private data (call history, billing, tickets) or changing their account, the bot must verify their identity by asking for their national ID number (NIC) and checking it against the account on file. A phone number alone is never accepted as proof - this stops someone from pulling a stranger's data just by knowing or guessing their number.
6. The reply is sent back in whichever language the customer used - English, Sinhala, or Tamil are all supported, including romanized versions typed in English letters.

6. Multi-tenancy and the Nexus Telecom example

Every client's data is kept completely separate ("tenant isolation") even though they all run on the same platform. Nexus Telecom is the most fully-built example client: it has a realistic mobile-operator database (customers, phone lines, call records, bills, support tickets) - the kind of data a real telecom company would actually have.

The key design idea: the chatbot and the human staff portal both read and write the SAME database. So if the bot recharges a customer's balance, a staff member looking at the portal sees that exact change immediately, and vice versa. It is not two separate demo systems pretending to be connected - it is genuinely one shared system.

7. Tech stack, in simple terms

- Frontend: React (JavaScript)
- Backend: Python, FastAPI
- Database: SQL (SQLite for this demo; can move to a bigger database like Postgres for real scale)
- Document search: FAISS (a vector similarity search library) plus a multilingual embedding model
- AI: Google Gemini and Groq (Llama models) - swappable
- Voice: speech-to-text and text-to-speech for live voice calls
- Deployment: the backend runs in a Docker container on DigitalOcean; the frontend is hosted on Render; both auto-deploy whenever code is pushed to GitHub

8. A few honest, practical lessons

- The AI document-search models need real memory (roughly half a gigabyte to a gigabyte) just to load
 - that is not obvious until you actually try to deploy on a small or free server. A good real-world example of "it works on my machine" meeting actual hosting constraints.
- Security should never be just a polite instruction to the AI ("please ask for ID first") - it has to be enforced in real backend code, because an AI model cannot be fully guaranteed to always follow instructions. The identity check here happens in the backend function itself, not only as a suggestion in the prompt.

9. Likely interview questions, with short answers

Q: What is RAG, and why not just use ChatGPT directly?

A: RAG (Retrieval-Augmented Generation) means the system searches the company's own real documents for the answer FIRST, then asks the AI to phrase a reply using only that retrieved information. Without this, an AI model can confidently invent a wrong price or policy ("hallucinate") because it is just guessing from general training data, not the company's actual current facts.

Q: How do you keep different companies' data separate?

A: Every record in the database is tagged with a client ID, and every query is filtered by it. Each company also gets its own document search index. So even though it is one shared platform, no client can ever see another client's data.

Q: How does the bot decide when to search documents versus take an action?

A: The AI model is given a list of available "tools" (functions) it can call, like "search knowledge base" or "check billing". Based on what the customer asked, the model decides which tool fits and calls it with the right details. The actual data lookup or action always happens in real backend code, never invented by the AI.

Q: How do you stop the bot from leaking someone's private data?

A: Any tool that reveals or changes a specific customer's data requires their phone number AND their national ID number, and the backend code checks that ID against the real customer record before returning anything - even if the AI forgets to ask, the code blocks it regardless.

Q: How do you support multiple languages?

A: The knowledge base documents are kept in English, but the system detects the customer's language and translates their question into English just for the search step, then instructs the AI to reply back in the customer's own language - English, Sinhala, or Tamil, including romanized versions.

Q: Where does this actually run, and how is it deployed?

A: The backend runs in a Docker container on DigitalOcean (it needed more memory than free-tier hosts could offer, because of the AI document-search models), and the frontend is a static site on Render. Both are set up to auto-deploy straight from GitHub on every push.

Q: What would you improve if you kept building this?

A: Add persistent storage so demo data survives restarts, move from SQLite to a production database like Postgres, add proper one-time-password verification instead of ID-number matching, and add monitoring and alerting for production use.